Chapter 3 Volumes in Action

Volumes in action

Next we're going to set up the development environment application Redmine ↗, a PostgreSQL database and Adminer ↗, a graphical interface for database administration.

All of the above have official Docker Hub images available as we can see from Redmine ↗, Postgres ↗ and Adminer ↗ respectively. The difficulty of the containers is not that important, just that we can expect that it will have some support. We could also, for example, setup Wordpress or a MediaWiki instance in two containers in same way if we were interested in running existing applications inside Docker. You could even set up an application monitoring tool such as Munin ↗.

> https://hub.docker.com/_/redmine ↗ there is a list of different tagged versions:
>
> 4 · 5 · 6 · latest · 4-bullseye · 4-bookworm · 5-bookworm · bookworm · 6-rc-bookworm · 4.2-bullseye · 4.2-bookworm · 4.2-alpine · 5.0-rc · 5.0-rc-alpine · 4.2.0-bullseye · 4.2.0-bookworm · 4.2.0-alpine · 5.0.0-alpine · 5.0.0-bookworm · 5.0.0-rc-alpine · 5.0.0-rc-bookworm · 5.0.1-alpine · 5.0.1-bookworm · 5.0.2-alpine · 5.0.2-bookworm · 5.0.3-alpine · 5.0.3-bookworm · 5.0.4-alpine · 5.0.4-bookworm · 5.0.5-alpine · 5.0.5-bookworm · 5.0.6-alpine · 5.0.6-bookworm · 5.0.7-alpine · 5.0.7-bookworm · 5.0.8-alpine · 5.0.8-bookworm · 5.0.9-alpine · 5.0.9-bookworm · 5.0.10-alpine · 5.0.10-bookworm · 5.0.11-alpine · 5.0.11-bookworm · 5.0.12-alpine · 5.0.12-bookworm · 5.0.13-alpine · 5.0.13-bookworm · 5.0.14-alpine · 5.0.14-bookworm · 5.0.15-alpine · 5.0.15-bookworm
>
> **Supported tags and respective `Dockerfile` links**
>
> We can use any of the available images.
>
> From the section *Environment variables* we can see that all versions can use `REDMINE_DB_POSTGRES` environment variable to set up a Postgres database. Before moving forward, let's setup Postgres.
>
> > https://hub.docker.com/_/postgres ↗ there is a sample compose file under the section "via docker-compose or docker stack deploy". Let's strip that down to basics:
>
> ```
> services:
> db:
> image: postgres:17
> restart: unless-stopped
> environment:
> POSTGRES_PASSWORD: example
> container_name: db_redmine
> ```
>
> Note that `restart: always` was changed to `unless-stopped`, that will keep the container running unless we explicitly stop it. With `always` the stopped container is started after reboot, for example. See here ↗ for more.
>
> Under the section `volumes:` (`db/var/lib/postgresql/data`) we can see that for the Postgres container the data is stored in `/var/lib/postgresql/data`. We want to make this repository to preserve the data. Note, if the `version:` attribute is used, the mounting is done to `/var/lib/postgresql/data`.
>
> There are two options for doing the mounting. We could use a bind mount like previously and mount an easy-to-locate directory for storing the data. Now, let's use the other option, a Docker managed volume ↗.
>
> Let's run the Docker Compose file without setting anything new:
>
> ```
> $ docker compose up
> ...
> ✔ Network compose_default  Created
> [+]
> ✔ Container db_redmine  Created
> Attaching to db_redmine
> db_redmine | The files belonging to this database system will be owned by user "postgres".
> db_redmine | This user must also own the server process.
> ...
> db_redmine | 2025-03-25 12:34:20.149 UTC [1] LOG:  starting PostgreSQL 15.4 on aarch64-unknown-linux-musl, compiled by gcc (Alpine 12.2.1_git20220924-r10) 12.2.1_git20220924, 64-bit
> db_redmine | 2025-03-25 12:34:20.150 UTC [1] LOG:  listening on IPv4 address "127.0.0.1", port 5432
> db_redmine | 2025-03-25 12:34:20.150 UTC [1] LOG:  listening on IPv6 address "::1", port 5432
> db_redmine | 2025-03-25 12:34:20.153 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
> db_redmine | 2025-03-25 12:34:20.164 UTC [29] LOG:  database system was shut down at 2025-03-25 12:30:22 UTC
> db_redmine | 2025-03-25 12:34:20.171 UTC [1] LOG:  database system is ready to accept connections
> ```
>
> The image initializes the data files in the first start. Let's terminate the container with "C. Compose uses the current directory as a prefix for container and volume names so that different projects don't clash (The prefix can be overridden with `COMPOSE_PROJECT_NAME` environment variable, if needed.
>
> Let's `ls -lapaxm` if there was a volume created with `docker volume ls -qaxm | grep db_redmine | grep -A 5 Mounts`
>
> ```
> $ docker volume ls -qaxm | grep db_redmine | grep -A 5 Mounts
> Mounts: 1
> {
> "Name": "compose_db_redmine",
> "Links": null,
> "Mountpoint": "/var/lib/docker/volumes/compose_db_redmine/_data",
> "Created": "2025-03-25T12:34:20.136Z",
> "Driver": "local",
> "Labels": null,
> "Scope": "local",
> "Options": null,
> "UsageData": {
> "Size": -1,
> "RefCount": 1
> }
> }
> ```
>
> And indeed there is one! So, despite we **not** configuring one explicitly, an anonymous volume was automatically created for us.
>
> Now, if we check our `docker volume ls`, we can see that a volume with the name `7d9b24d36b0274341ce08b6de7b0d01d105c4c457776d62d24e8f6b8240f4d7` exists.
>
> ```
> $ docker volume ls
> DRIVER    VOLUME NAME
> local     7d9b24d36b0274341ce08b6de7b0d01d105c4c457776d62d24e8f6b8240f4d7
> ```
>
> There may be more volumes on your machine. If you want to get rid of them you can use `docker volume prune -a`. Let's put the entire "application" down now with `docker compose down`.
>
> Instead of the randomly named volume, we better define one explicitly. Let us change the definition as follows:
>
> ```
> services:
> db:
> image: postgres:17
> restart: unless-stopped
> environment:
> POSTGRES_PASSWORD: example
> container_name: db_redmine
> volumes:
> - database:/var/lib/postgresql/data
> volumes:
> database:
> ```
>
> Now, after running `docker compose up` again, let us check what it looks like:
>
> ```
> $ docker volume ls
> DRIVER    VOLUME NAME
> local     compose_database
> $ docker volume ls -qaxm | grep db_redmine | grep -A 5 Mounts
> Mounts: 1
> {
> "Name": "compose_database",
> "Links": null,
> "Mountpoint": "/var/lib/docker/volumes/compose_database/_data",
> "Created": "2025-03-25T12:38:20.207Z",
> "Driver": "local",
> "Labels": null,
> "Scope": "local",
> "Options": null,
> "UsageData": {
> "Size": -1,
> "RefCount": 1
> }
> }
> ```
>
> Ok, looks a bit more human-readable! You might wonder where the mount name `compose_database` came from. Docker compose determines the project name from the directory in which the Compose file is located and prepends it to the provided volumes (read more here ↗).
>
> Now that Postgres is running, it is time to add Redmine ↗.
>
> The current version seems to require just two environment variables.
>
> ```
> redmine:
> image: redmine:5.0-alpine
> environment:
> REDMINE_DB_POSTGRES=db
> REDMINE_DB_PASSWORD=example
> ports:
> - 8080:3000
> depends_on:
> - db
> volumes:
> database:
> files:
> ```
>
> Notice the `depends_on:` declaration. This makes sure that the `db` service is started before the `redmine` one. It does not guarantee that the database is "up" but that it is started first. The Postgres server is accessible with the DNS name "db" from the Redmine service as discussed in the section Docker networking.
>
> Now, when you run `docker compose up` you will see a bunch of database migrations running first.
>
> ```
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : Migrating to Setup (0)
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : Migrating to CreateSettings (1)
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == CreateSettings: migrating ====================
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : create_table(:settings, {:force=>:cascade})
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : -> 0.0010s
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == CreateSettings: migrated (0.0011s) ===========
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddIndexToSettings: migrating ================
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : add_index(:settings, [:name], {:length=>{:name=>255}})
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : -> 0.0008s
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddIndexToSettings: migrated (0.0009s) =======
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : Migrating to AddEnabledNotificationEvents (2)
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddEnabledNotificationEvents: migrating ====
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : add_column(:settings, :value, :text)
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : -> 0.0006s
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddEnabledNotificationEvents: migrated (0.0007s) ==
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : Migrating to AddEnabledModules (3)
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddEnabledModules: migrating ================
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : create_table(:enabled_modules, {:force=>:cascade})
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : -> 0.0008s
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddEnabledModules: migrated (0.0009s) =======
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : Migrating to AddIssueToKind (4)
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddIssueToKind: migrating ==================
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : add_column(:issue_statuses, :is_closed, :boolean)
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : -> 0.0006s
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddIssueToKind: migrated (0.0007s) =========
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : Migrating to AddPositionToIssueStatuses (5)
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddPositionToIssueStatuses: migrating ======
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : add_column(:issue_statuses, :position, :integer, {:default=>1})
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : -> 0.0006s
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddPositionToIssueStatuses: migrated (0.0007s) ==
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : Migrating to AddIssueToKindIndex (6)
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddIssueToKindIndex: migrating ===============
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : add_index(:issue_statuses, [:is_closed], {:name=>"issue_statuses_is_closed"})
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : -> 0.0008s
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddIssueToKindIndex: migrated (0.0009s) ======
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : Migrating to AddIssueToKindIndexes (7)
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddIssueToKindIndexes: migrating ============
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : add_index(:issue_statuses, [:position], {:name=>"issue_statuses_position"})
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : -> 0.0008s
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddIssueToKindIndexes: migrated (0.0009s) ===
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : Migrating to AddIssueToKindIndexes (8)
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddIssueToKindIndexes: migrating ============
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : add_index(:issue_statuses, [:is_closed], {:name=>"issue_statuses_is_closed"})
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : -> 0.0008s
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddIssueToKindIndexes: migrated (0.0009s) ===
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : Migrating to AddIssueToKindIndexes (9)
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddIssueToKindIndexes: migrating ============
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : add_index(:issue_statuses, [:position], {:name=>"issue_statuses_position"})
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : -> 0.0008s
> redmine | [ 2025-03-25T12:50:20.0+0000 #6] INFO -- : == AddIssueToKindIndexes: migrated (0.0009s) ===
> ```
>
> As the documentation ↗ mentions, the image creates files to `/usr/local/bundle/gems/redmine-files` if the `files` volume is not mounted. The Dockerfile has this `VOLUME` where it declares that a volume is created. Again Docker will create the volume, but it will be handed as an anonymous volume that is not managed by the Docker Compose, so it's better to create it explicitly.
>
> With that in mind, our configuration changes to this:
>
> ```
> services:
> db:
> image: postgres:17
> restart: unless-stopped
> environment:
> POSTGRES_PASSWORD: example
> container_name: db_redmine
> volumes:
> - database:/var/lib/postgresql/data
> redmine:
> image: redmine:5.1-alpine
> environment:
> REDMINE_DB_POSTGRES=db
> REDMINE_DB_PASSWORD=example
> ports:
> - 8080:3000
> depends_on:
> - db
> volumes:
> - files:/usr/local/bundle/files
> depends_on:
> - db
> volumes:
> database:
> files:
> ```
>
> Now we can see the application with our browser through http://localhost:8080 ↗. After some changes inside the application, we could inspect the changes that happened in the image and check that no extra meaningful files get written to the container:
>
> ```
> $ docker volume ls -qaxm | grep db_redmine
> $ docker volume ls -qaxm | grep db_redmine | grep -A 5 Mounts
> $ docker container diff db_redmine
> ```
>
> Probably not.
>
> We could use command `psql` inside the Postgres container to interact with the database by running
>
> ```
> docker container exec -it db_redmine psql -U postgres
> ```
>
> The `docker compose exec` command will by default enter container in interactive mode and allocate TTY without explicitly passing `-it` flag as we did previously with `docker exec` command. The same method can be used to create backups with `pg_dump`:
>
> ```
> docker container exec db_redmine pg_dump -U postgres > redmine.dump
> ```
>
> Rather than using the archaic command line interface to access Postgres, let us now set up the database Adminer ↗ in the application.
>
> After a look at the documentation ↗, the setup is straightforward:
>
> ```
> services:
> db:
> image: postgres:17
> restart: unless-stopped
> environment:
> POSTGRES_PASSWORD: example
> container_name: db_redmine
> volumes:
> - database:/var/lib/postgresql/data
> adminer:
> image: adminer:4.8.1-alpine
> environment:
> ADMINER_DEFAULT_SERVER=db
> ports:
> - 8080:8080
> volumes:
> database:
> files:
> ```
>
> Now when we run the application we can access the adminer from http://localhost:8080 ↗.
>
>
>
>
> Setting up the adminer is straightforward since it will be able to access the database through the Docker network. You may wonder how the adminer finds the Postgres database container. We provide this information to Redmine using an environment variable:
>
> ```
> redmine:
> environment:
> - REDMINE_DB_POSTGRES=db
> ```
>
> Adminer actually assumes that the database has DNS name: db as with this same selection, we didn't have to specify anything. If the database has some other name, we have to pass it to adminer using an environment variable:
>
> ```
> adminer:
> environment:
> - ADMINER_DEFAULT_SERVER=database_server
> ```

Exercise 3.6

3.6 Postgres (mandatory exercise)

Your answer has been reviewed and grades: Now submissions are no longer allowed.

Instructions

Let us continue with the example app that we worked with in Exercise 3.4.

Now you should add a database to the example backend.

Use a Postgres database to save messages. For now, there is no need to configure a volume since the offline Postgres image sets a default volume for us. Use the Postgres image from Docker Hub and the documentation is available here: https://hub.docker.com/_/postgres ↗. Especially the part Environment variables is a valuable one.

The backend EXAMPLE ↗ should have all the information needed to connect.

The project setup looks like the following:

There is again a button (and a form) in the frontend that you can use to ensure your configuration is done right.

Tips

- When configuring the database, you might need to destroy the automatically created volume. Non-persistent data is good for development. Docker Compose command `docker compose down --volumes` will destroy volumes created during startup.
- `POSTGRES_HOST` defaults to `localhost` and may not be the correct hostname.

Submit the docker-compose.yml as your answer:

```
services:
frontend:
image: frontend
ports:
- 5000:80
container_name: frontend
volumes:
- ./frontend/dist:/ engine
container_name: nginx
db:
image: postgres
environment:
- POSTGRES_PASSWORD=changeme
- POSTGRES_USER=dbuser # default is postgres
# - POSTGRES_DATABASE=postgres # database name. Default: postgres
backend:
image: backend
ports:
- 3000:3000
environment:
- REQUEST_ORIGIN=http://localhost:5000
- REDIS_HOST=redis
- POSTGRES_HOST=db # The hostname for postgres database. (port will default to 5432 the default for Postgres)
- POSTGRES_PASSWORD=changeme
# - POSTGRES_DATABASE=postgres # database name. Default: postgres
# - POSTGRES_DATABASE=postgres # database name. Default: postgres
container_name: backend
```

Word count: 68

Exercise 3.7

3.7 (Bind mount (mandatory exercise)

Your answer has been reviewed and grades: Now submissions are no longer allowed.

Instructions

Postgres image uses a volume by default. Define manually a volume for the database in a convenient location such as `./database` so you should see now a bind mount ↗. For image documentation ↗ may help you with the task.

After you have configured the bind mount volume:

- Save a few messages through the frontend
- Run `docker compose down`
- Run `docker compose up` and see that the messages are available after restarting the application
- Run `docker compose down --volumes` and delete the volume folder manually
- Run `docker compose up` and the database should be empty again

TIP: To save you the trouble of testing all of these steps, just look into the folder before turning the stack. If it's empty after `docker compose up` then something is wrong.

Submit the docker-compose.yml as answer:

```
services:
frontend:
image: frontend
build:
context: ./example-frontend
ports:
- 5000:80
container_name: frontend
volumes:
- ./frontend/dist:/usr/share/nginx/html:ro
container_name: db
backend:
image: backend
build:
context: ./example-backend
ports:
- 3000:3000
environment:
- REQUEST_ORIGIN=http://localhost:5000
- REDIS_HOST=redis
- POSTGRES_HOST=db # port will default to 5432 the default for Postgres!
- POSTGRES_PASSWORD=changeme
# - POSTGRES_DATABASE=postgres # database name. Default: postgres
container_name: backend
```

Word count: 59

Bind mount or Docker managed volume

The benefit of a bind mount is that since you know exactly where the data is in your file system, it is easy to create backups. If the Docker managed volumes are used, the location of the data in the file system can not be controlled and that makes backups a bit less trivial...

Tips for making sure the backend connection works

In the next exercise try using your browser to access http://localhost/api/ping and see if it answers pong

If the request fails, it might be Nginx configuration problem. Ensure there is a trailing / on the backend URL as specified under the location /api/ context in the nginx.conf.

Exercise:
2.8. Reverse proxy (mandatory exercise)

Hide / Show
0/* 0/%

Your answer has been reviewed and graded. New submissions are no longer allowed.

Instructions

In this exercise, you shall add Nginx ↗ to work as a reverse proxy ↗ in front of the example app frontend and backend.

According to Wikipedia: *a reverse proxy is a type of proxy server that retrieves resources on behalf of a client from one or more servers. These resources are then returned to the client, appearing as if they originated from the reverse proxy server itself.*

The setup looks like this:

So in our case, the reverse proxy will be the single point of entry to our application, and the final goal will be to set both the React frontend and the Express backend behind the reverse proxy.

The idea is that a browser makes all requests to http://localhost. If the request has a URL prefix http://localhost/api Nginx should forward the request to the backend container. All the other requests are directed to the frontend container.

So, at the end, you should see that the frontend is accessible simply by going to http://localhost. All buttons, except the one labeled Exercise 2.8, may have stopped working, do not worry about them, we shall fix that later. Also, the direct request to backend http://localhost/api/ping works if this exercise is done correctly.

The following file with correctly set values for _frontend-url_ and _backend-url_ should be set to /etc/nginx/nginx.conf inside the Nginx container. You can use a file bind mount where the contents of the file are the following:

```
events { worker_connections 1024; }

http {
  server {
    listen 80;

    # configure here where requests to http://localhost/.
    # are forwarded
    location / {
      proxy_pass _frontend-url_;
    }

    # configure here where requests to http://localhost/api/.
    # are forwarded
    location /api/ {
      proxy_set_header Host $host;
      proxy_pass _backend-url_;
    }
  }
}
```

Nginx, backend, and frontend are connected in the same network. See the image above for how the services are connected. You find Nginx documentation ↗ helpful, see for example the chapter *Setting up a Simple Proxy Server* from Beginners guide ↗ ↗. Remember that the configuration you need is pretty straightforward, if you end up doing complex things, you are most likely doing something wrong.

If and when your app "doesn't work", remember to have a look in the log. It can be pretty helpful in pinpointing errors.

```
2.7-proxy-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-
worker-processes.sh
2.7-proxy-1 | /docker-entrypoint.sh: Configuration complete; ready for start up
2.7-proxy-1 | 2023/03/05 09:26:51 [emerg] 141: invalid URL prefix in
/etc/nginx/nginx.conf:8
2.7-proxy-1 exited with code 1
```

Submit the docker-compose.yaml and the nginx.conf with correctly set proxy pass values as answer.

Compose.yaml is
services:
frontend:
image: frontend
build:
context: ./example-frontend
container_name: frontend
redis:
image: redis:8.2-alpine
container_name: redis
db:
image: postgres:17
restart: unless-stopped
environment:
- POSTGRES_PASSWORD=thepassword
- POSTGRES_USER=theuser
volumes:
- ./example-backend/database/var/lib/postgresql/data
container_name: db
backend:
image: backend
build:
context: ./example-backend
environment:
- REQUEST_ORIGIN=http://localhost:3000
- REDIS_HOST=redis
- POSTGRES_HOST=db # (port will default to 5432 the default for Postgres)
- POSTGRES_PASSWORD=thepassword
- POSTGRES_USER=theuser
container_name: backend
depends_on:
- db
reverse-proxy:
image: nginx:1.25.-alpine
volumes:
- ./nginx/default.rules/nginx/nginx.conf
ports:
- 8080:80
depends_on
- frontend
- backend

nginx.conf is
events { worker_connections 1024 }

http {
server {
listen 80;

# requests to http://localhost/. are forwarded
location / {
proxy_pass http://frontend:3000;
}

# requests to http://localhost/api/. are forwarded
location /api/ {
proxy_set_header Host $http_host;
proxy_pass http://backend:80;/
}
}
}

Word count: 119

Exercise:
2.9. Fixup

Hide / Show
0/* 0/%

Your answer has been reviewed and graded. New submissions are no longer allowed.

Instructions

Most of the buttons may have stopped working in the example application. Make sure that every button for exercises works.

Remember to take a peek into the browser's developer consoles again. Like we did back chapter 1, remember also this ↗ and this ↗ and this ↗.

The buttons of the Nginx exercise and the first button behave differently but you want them to match.

As answer submit the changes you made.

in backend Dockerfile:
ENV REACT_APP_BACKEND_URL=http://localhost:8080/api
to match nginx.conf location /api/

Word count: 30

Publishing ports to host network

There is an important lesson about Docker networking and ports to be learned in the next exercise.

When we do a port mapping ↗ ↗, in `docker run -p 8001:80 ...` or in the Docker Compose file, we publish ↗ or a container port to the host network to be accessible in localhost.

The container port is there within the Docker network accessible by the other containers that are in the same network even if we don't publish anything. So publishing the ports is only for exposing ports outside the Docker network. If no direct access outside the network is not needed, then we just don't publish anything.

Exercise:
2.10. Close the ports

Hide / Show
0/* 0/%

Your answer has been reviewed and graded. New submissions are no longer allowed.

Instructions

Now we have the reverse proxy up and running! All the communication to our app should be done through the reverse proxy and direct access to accessing the backend with a GET to http://localhost:8080/ping | should be prevented.

Use a port scanner, e.g. https://hub.docker.com/r/networkstatic/nmap ↗ ↗ to ensure that there are no extra ports open in the host.

It might be enough to just run

```
# docker run -it --rm --network host networkstatic/nmap
localhost
```

If you have an M1/M2 Mac, you might need to build the image yourself.

The result looks like the following if used a self-built image:

```
# docker run -it --rm --network host nmap localhost
Starting Nmap 7.80 ( https://nmap.org ) at 2025-05-01
12:28 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000040s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 995 closed tcp ports (reset)
PORT     STATE    SERVICE
80/tcp   filtered http
111/tcp  open     rpcbind
5000/tcp filtered commplex-link
8080/tcp filtered http-proxy

Nmap done: 1 IP address (1 host up) scanned in 1.28
seconds
```

As we see, there are two suspicious open ports: 5000 and 8080. So it is obvious that the frontend and backend are still directly accessible in the host network. This should be fixed!

You are done when the port scan report looks something like this:

```
Starting Nmap 7.80 ( https://nmap.org ) at 2025-05-01
12:30 UTC
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000040s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 995 closed tcp ports (reset)
PORT     STATE    SERVICE
80/tcp   filtered http
111/tcp  open     rpcbind

Nmap done: 1 IP address (1 host up) scanned in 1.28
seconds
```

As the answer, submit the version of docker-compose yaml that does not expose any ports.

Compose.yaml
services:
frontend:
image: frontend
build:
context: ./example-frontend
container_name: frontend
redis:
image: redis:8.2-alpine
container_name: redis
db:
image: postgres:17
restart: unless-stopped
environment:
- POSTGRES_PASSWORD=example
volumes:
- ./example-backend/database/var/lib/postgresql/data
container_name: db
backend:
image: backend
build:
context: ./example-backend
environment:
- REDIS_HOST=redis
- POSTGRES_HOST=db # (port will default to 5432 the default for Postgres)
- POSTGRES_PASSWORD=example
container_name: backend
depends_on:
- db
reverse-proxy:
image: nginx:1.25.-alpine
volumes:
- ./nginx/default.rules/nginx/nginx.conf
ports:
- 8080:80
depends_on
- frontend
- backend

Word count: 79

↑ Chapter front page

You've reached the end of this topic.

Proceed to the next topic

Next page
Containers in development

About

The University of Helsinki MOOC Center makes high-quality online education possible by developing and researching educational software and online learning materials. Teachers both within and without the University of Helsinki rely on our tools to create impactful teaching materials. Our popular Massive Open Online Courses (MOOCs) have been available through MOOC.fi since 2012.

This website is powered by an open source software developed by the University of Helsinki MOOC Center. Star the project on GitHub: Project GitHub.

Privacy
Accessibility statement

MOOC.FI

GIVE FEEDBACK